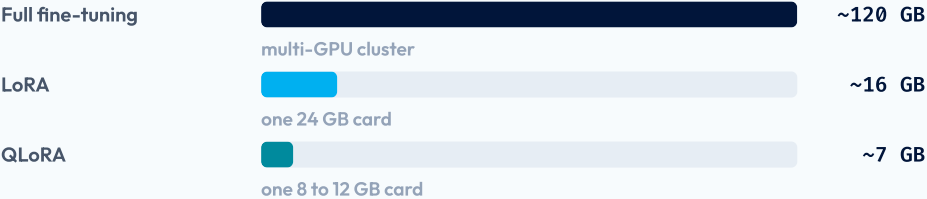


LoRA and QLoRA

Explained Visually

How fine-tuning a 7B model went from needing a datacentre to fitting on a laptop GPU.

MEMORY TO FINE-TUNE THE SAME 7B MODEL



Same model. Same task. Three memory footprints.



The Weights Are The Smallest Part

Fine-tuning a 7B model with Adam, approximate breakdown

WHERE THE MEMORY ACTUALLY GOES

Model weights fp16		14 GB
Gradients		14 GB
Adam state 1		28 GB
Adam state 2		28 GB
fp32 master copy		28 GB
Activations		varies

TOTAL

~112 GB AND UP

Navy is the model. Red is everything training adds on top.

🔑 THE REALISATION BEHIND LORA

Gradients, optimizer states and master weights all scale with the number of **trainable** parameters. Make almost nothing trainable and almost all of that memory disappears.

→ The 14 GB of frozen weights can then stay in a compact form. **That single observation is the whole idea.**



AI Coach John

[Follow](#)

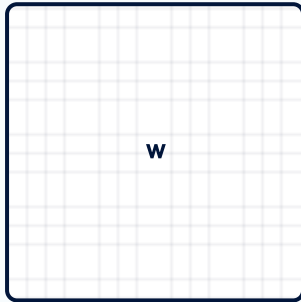


[Repost](#)

What A Weight Matrix **Actually** Is

One attention projection inside a 7B model

A SINGLE PROJECTION MATRIX



4096 rows

16.7M

NUMBERS IN THIS ONE MATRIX

4096 columns

PER LAYER

Query, key, value and output projections, plus much larger feed-forward matrices

PER MODEL

Hundreds of matrices like this one, repeated in every layer

FINE-TUNING

Computing a gradient for, and updating, **every single one of those numbers**



Hold that picture. The next two slides only work if this one is clear.



AI Coach John

[Follow](#)

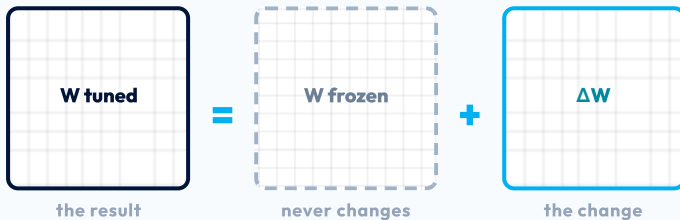


[Repost](#)

You Learn A Change, Not A Matrix

Fine-tuning produces W plus a delta, and the delta is simple

WHAT FINE-TUNING PRODUCES



🔗 THE OBSERVATION

The change induced by adapting to a narrow task has low intrinsic rank. Its information content is far smaller than its 16.7 million entries suggest.

💡 THE INTUITION

Teaching a general model to write in your support team's tone is a small, structured adjustment. It does not need 16.7M independent degrees of freedom to express.

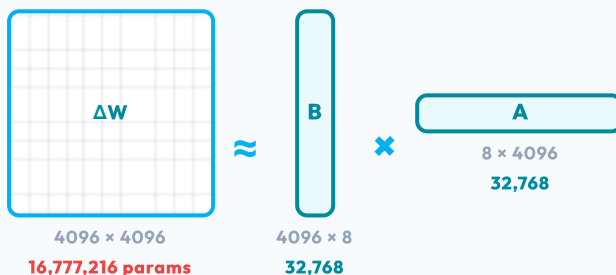
✂ So do not store ΔW as a full matrix. **Factorise it.**



One Big Matrix Becomes Two Thin Ones

B times A has the same shape, at a fraction of the parameters

INSTEAD OF LEARNING ΔW DIRECTLY



👉 16,777,216 → 65,536 · a 99.6% cut

Rank $r = 8$. Thin dimensions shown larger than scale so they stay visible.

✓ WHY THIS IS ALLOWED

The product $B \times A$ is still 4096×4096 , the right shape to add to W . It simply cannot express arbitrary matrices, only low-rank ones, **which is exactly what we argued ΔW is.**



AI Coach John

Follow

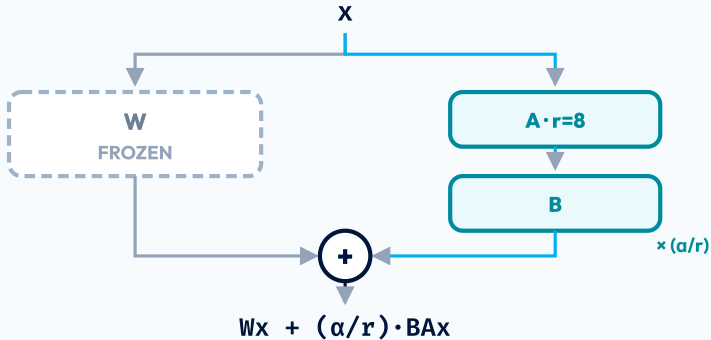


Repost

How The Adapter Plugs In

Two paths, summed. The base path is never touched.

FORWARD PASS WITH LORA



Grey path is frozen. Only A and B receive gradients.

PYTHON

```
class LoRALinear(nn.Module):
    def __init__(self, base, r=8, alpha=16, dropout=0.05):
        super().__init__()
        self.base = base
        for p in self.base.parameters():
            p.requires_grad = False  # freeze it
        d_in, d_out = base.in_features, base.out_features
        self.A = nn.Parameter(torch.randn(r, d_in) * 0.01)
        self.B = nn.Parameter(torch.zeros(d_out, r))  # ZEROS
        self.scaling = alpha / r
        self.drop = nn.Dropout(dropout)

    def forward(self, x):
        lora = self.drop(x) @ self.A.T @ self.B.T * self.scaling
        return self.base(x) + lora
```



Why B Starts At Zero

A small detail that makes training stable

AT INITIALISATION

A = small random values

B = all zeros

$$B \times A = 0$$

$$\text{output} = Wx + (\alpha/r) \cdot 0 = Wx$$

= The adapted model starts **identical** to the base model

× IF BOTH WERE RANDOM

The model would start as base plus noise. Degraded before step one, and the first stretch of training would go on undoing that damage.

× IF BOTH WERE ZERO

The gradient of a product where both factors are zero is zero. Nothing would ever learn. One side has to be non-zero to break the symmetry.



Training therefore begins from a **known good state** and moves away from it gradually.



AI Coach John

Follow

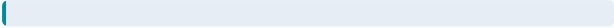
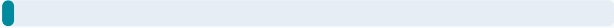
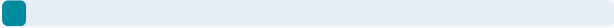
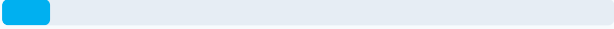
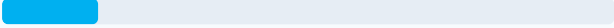
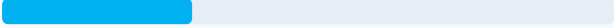


Repost

Rank r, The Main Dial

How much capacity the adapter is given

PARAMETERS PER 4096 × 4096 MATRIX

r=1		8,192 · 0.05%
r=4		32,768 · 0.2%
r=8		65,536 · 0.4%
r=16		131,072 · 0.8%
r=32		262,144 · 1.6%
r=64		524,288 · 3.1%
r=128		1.05M · 6.3%
full		16.8M · 100%

R = 4 TO 8

Style, tone, output format. Often enough, and people habitually overestimate what they need.

R = 16 TO 32

Domain adaptation and task specific reasoning. A sensible default starting point.

R = 64 TO 128

Larger behavioural shifts, or when lower ranks visibly plateau below target.



Higher rank is not automatically better. It adds capacity to **overfit a small dataset** and costs memory. Start low, raise only if validation says so.



AI Coach John

Follow



Repost

Alpha, And What Scaling Controls

The update is multiplied by alpha divided by r

THE SCALAR IN FRONT OF THE ADAPTER

$$\text{output} = Wx + \left(\frac{\alpha}{r} \right) \cdot \text{BA}x$$

Raise r from 8 to 16 and BA produces larger values simply because more terms are summed. Dividing by r keeps the update's magnitude stable, **so you can change r without retuning the learning rate.**

SETTING	SCALING	CHARACTER
ALPHA = R	1.0	Conservative, gentle adaptation
ALPHA = 2R	2.0	The most widely used default
ALPHA = 4R	4.0	Stronger adaptation, less stable


r = 16 with α = 32 gives scaling 2.0



Always report alpha with r. "alpha = 32" alone is meaningless: mild at r = 32, aggressive at r = 8.



Where You Attach The Adapters

Placement matters as much as rank

ONE TRANSFORMER BLOCK

ATTENTION

q_proj

ADAPT, ALMOST ALWAYS

k_proj

ADAPT, OFTEN

v_proj

ADAPT, ALMOST ALWAYS

o_proj

ADAPT, OFTEN

FEED-FORWARD · HOLDS MOST OF THE PARAMETERS

gate_proj

BIGGER SHIFTS

up_proj

COSTS MEMORY

down_proj

COSTS MEMORY

PYTHON

Conservative, query and value only

```
LoraConfig(r=8, lora_alpha=16,  
           target_modules=["q_proj", "v_proj"])
```

Broader, all attention projections

```
LoraConfig(r=16, lora_alpha=32,  
           target_modules=["q_proj", "k_proj", "v_proj", "o_proj"])
```

Broadest, attention plus feed-forward

```
LoraConfig(r=16, lora_alpha=32,  
           target_modules=["q_proj", "k_proj", "v_proj", "o_proj",  
                           "gate_proj", "up_proj", "down_proj"])
```



Module names differ by architecture. Print the model and read the real names first: a typo silently adapts nothing.



AI Coach John

Follow



Repost

Why The Savings Are So Large

7B model with Adam, approximate figures

Component	Full FT	LoRA
Base Weights FP16	14 GB	14 GB
Gradients	14 GB	0.05 GB
Optimizer State 1	28 GB	0.1 GB
Optimizer State 2	28 GB	0.1 GB
FP32 Master Copy	28 GB	0.1 GB
Total	~112 GB	~15 to 18 GB

Where The Saving Comes From

It is **not** the weights, which are still fully loaded. It is that gradients, optimizer states and master weights scale with **trainable** parameters, and you just made about 0.5% of them trainable.



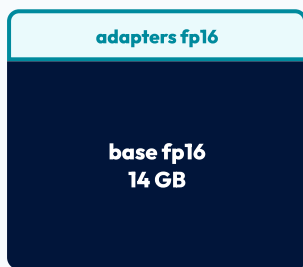
→ The 14 GB of frozen base weights is now the dominant cost. **That is exactly what QLoRA attacks.**



Compress The Frozen Part, **Keep Adapters Sharp**

Only what never learns gets quantized

SAME ADAPTERS, A MUCH SMALLER BASE



LORA

total ~15 to 18 GB



QLORA

total ~6 to 10 GB

Block heights are proportional to memory. The adapters are unchanged.

? WHY NOT QUANTIZE EVERYTHING

Gradient updates on 4-bit values would be far too coarse to learn from. The parts that actually learn stay in higher precision.



Quantization costs some precision in the base. **The full precision adapter absorbs much of it**, which is why this beats quantizing the whole model.



Sixteen Buckets Instead Of A Range

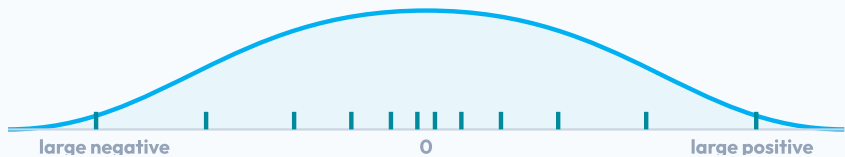
fp16 holds about 65,000 values. Four bits hold 16.

EVERY WEIGHT SNAPS TO ITS NEAREST BUCKET

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
fp16 value			stored as			bucket									
0.0312			→	0.0296			→	9							
-0.1847			→	-0.1875			→	3							
0.0009			→	0.0000			→	8							

4 bits per weight instead of 16, so a 4x cut, paid for by rounding.

NF4 • WHY THE BUCKETS ARE NOT EVENLY SPACED



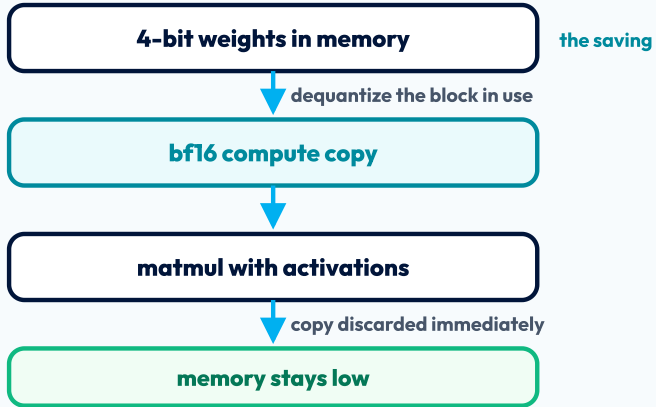
Weights cluster near zero, so evenly spaced buckets would waste most of them on empty tails. NF4 places buckets to match that distribution, **dense in the middle, sparse at the extremes**, so the same 16 buckets carry more information.



Stored In 4-Bit, Computed In 16

Small blocks are expanded only as they are used

ONE FORWARD PASS



The full model is never materialised in fp16.

↓ MEMORY

Dramatically lower. This is the entire reason QLoRA exists.

↑ COMPUTE

Some overhead on every pass, so training is typically slower per step than LoRA.



QLoRA buys memory, not speed. If the model already fits comfortably in your VRAM, plain LoRA trains faster.



AI Coach John

Follow



Repost

Two More Tricks QLoRA Uses

Double quantization, and paged optimizers

1 • DOUBLE QUANTIZATION

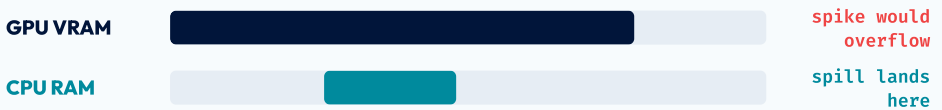
Quantizing needs one scaling constant per block of weights to map buckets back to real values. With small blocks, those constants add up.



Small per parameter, meaningful across billions of them.

2 • PAGED OPTIMIZERS

Memory spikes during training: a long sequence, a big batch, an unlucky moment. A spike past your VRAM normally kills the run.



Paged out when it spikes, paged back when the spike passes.

💓 The run **survives** instead of dying at hour six.



AI Coach John

Follow

PROTBIDGE

🔄 Repost

One 7B Model, Three Approaches

Approximate figures, varying by implementation

	FULL FT	LORA	QLORA
BASE WEIGHTS	fp16	fp16	NF4
BASE MEMORY	14 GB	14 GB	3.5 GB
TRAINABLE	7,000M	~4M	~4M
PERCENT TRAINABLE	100%	0.06%	0.06%
OPTIMIZER MEMORY	~84 GB	0.3 GB	0.3 GB
TOTAL VRAM	~112 GB	~16 GB	~6 to 10 GB
HARDWARE	multi-GPU	1 x 24 GB	1 x 8 GB
RELATIVE SPEED	baseline	fastest	slower
OUTPUT FILE	14 GB	~30 MB	~30 MB

THE LAST ROW IS EASY TO OVERLOOK

A LoRA adapter is a file of tens of megabytes. You can keep hundreds of them, version them in Git, ship them over a network and swap them at runtime. **None of that is practical with 14 GB checkpoints.**

The Code, Both Ways

Only the model loading differs

LORA

```
from peft import LoraConfig, get_peft_model

model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, torch_dtype=torch.bfloat16, device_map="auto")

config = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
    bias="none", task_type="CAUSAL_LM")

model = get_peft_model(model, config)
model.print_trainable_parameters()
# trainable: 4,194,304 || all: 6,742,609,920 || 0.0622%
```

QLORA

```
from transformers import BitsAndBytesConfig
from peft import prepare_model_for_kbit_training

bnb = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4", # the NF4 type
    bnb_4bit_compute_dtype=torch.bfloat16, # dequantize to this
    bnb_4bit_use_double_quant=True) # constants too

model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, quantization_config=bnb, device_map="auto")
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, config) # SAME config as above
```

= The adapter config, the training loop and the output artefact are identical. QLoRA is not a separate technique to learn.



What You Can Do With A 30 MB File

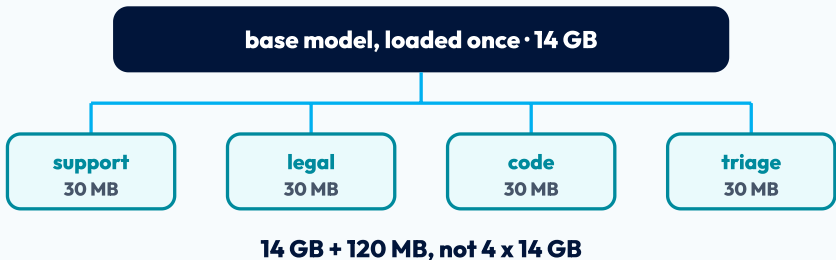
Merge it in, or keep it separate and hot-swap

PYTHON

```
# Merge into the base weights for deployment
merged = model.merge_and_unload()
merged.save_pretrained("./model-merged")
# A normal model, zero inference overhead:  $W_{new} = W + (a/r)BA$ 

# Or keep it separate and hot-swap
base = AutoModelForCausalLM.from_pretrained(MODEL_ID)
model = PeftModel.from_pretrained(base, "./adapters/support")
model.load_adapter("./adapters/legal", adapter_name="legal")
model.load_adapter("./adapters/code", adapter_name="code")
model.set_adapter("legal") # switch behaviour instantly
```

ONE BASE MODEL IN MEMORY, MANY BEHAVIOURS



Merging removes the ability to swap. Merge for one fixed behaviour, keep adapters separate when you serve several.



AI Coach John

Follow



Repost

When LoRA Falls Short

Honest limitations, and what to try in order

BIG SHIFTS

Updates are confined to a low rank subspace. Fine for narrow adaptation, where it even acts as regularisation. For genuinely new capability the constraint can bind.

HUGE DATASETS

With tens of thousands of good examples, the gap to full fine-tuning can widen. Adapter capacity becomes the limit rather than data.

4-BIT LOSS

QLoRA's base introduces precision loss. Negligible for most tasks, not for numerically sensitive work.

BAD RANK

Too low underfits and plateaus early and high. Too high overfits a small set. Sweep r on a subset first.

A PRACTICAL DEBUGGING LADDER

- 1 Check the **dataset** first, it is usually the cause
- 2 Raise r , 8 to 16 to 32, and retrain
- 3 Widen `target_modules` to include feed-forward layers
- 4 Switch QLoRA to LoRA to rule out quantization
- 5 Only then consider full fine-tuning



Common Mistakes

What goes wrong with LoRA and QLoRA

X CONFIG

- target module names that do not exist
- never calling `print_trainable_parameters`
- reporting alpha without `r`
- starting at `r=64` "to be safe"
- adapting only `q` and `v` for an FFN task

X RUNNING AND SHIPPING

- QLoRA when the model already fits
- skipping `prepare_model_for_kbit_training`
- merging when you needed to swap
- saving the merged model, not the adapter
- expecting LoRA to inject facts

SYMPTOM	LIKELY CAUSE
"loss does not move"	<code>target_modules</code> matched nothing
"OOM despite QLoRA"	batch size or sequence length, not weights
"plateaus early and high"	rank too low, or dataset too small
"overfits after one epoch"	rank too high for the data volume
"slower than plain LoRA"	expected, dequantization overhead
"adapter has no effect"	not loaded, or merged into the wrong base

Train 0.06%, Keep Most Of It

The whole method, in eight lines

THE MOVE	WHAT IT BUYS
$\Delta W \approx B \times A$	the whole idea, in one line
B STARTS AT ZERO	training begins from the base model
RANK R	adapter capacity, start low and raise
ALPHA OVER R	change r without retuning the learning rate
FREEZE THE BASE	optimizer memory collapses
4-BIT NF4 BASE	the frozen weights shrink about 4x
DEQUANTIZE ON THE FLY	memory saved, some speed paid
30 MB ADAPTERS	version them, ship them, swap them

ONE SENTENCE TO TAKE AWAY

The change fine-tuning makes to a weight matrix is far simpler than the matrix itself, so it can be stored as two thin matrices. That is what turned fine-tuning from a cluster operation into **one consumer GPU and a 30 MB file**.

 Follow **AI Coach John** for more visual AI engineering explainers.

